

1 WarehouseIQ — Flow Command: Comprehensive Architecture, Workflow, and Presentation Master Guide

Author: Manus AI

Project Name: smart-warehouse-operations

Live Application URL: <https://smartwarehouse-nwqedban.manus.space>

GitHub Repository: <https://github.com/saisudheernallabarika77-alt/Smart-Warehouse-Operations>

1.1 Executive Summary & Hackathon Context

Fulfillment centers operate under immense velocity, managing thousands of SKUs, incoming replenishment trucks, dynamic pick waves, and strict dispatch cut-off times. Traditional warehouse management systems (WMS) function as passive ledgers, recording inventory movements after they occur. **WarehouseIQ — Flow Command** represents a paradigm shift: an executive decision-driven smart warehouse operations platform that predicts bottlenecks, resolves stockout disputes through intelligent split-allocation algorithms, and guides warehouse shift leads with real-time decision intelligence.

This comprehensive master guide provides complete project transparency. It details the hackathon problem statement, our architectural solution, end-to-end data flows, tool and technology stacks, secure AI API design, codebase modularity, and a rigorous mentor Q&A briefing ensuring confident presentation performance.

1.2 1. Hackathon Problem Statement & Our Solution

1.2.1 The Core Challenge

Warehouses handle complex multi-item orders simultaneously. Poor inventory visibility, incorrect stock allocation, delayed picking, misplaced items, and fulfillment bottlenecks result in delayed shipments, severe stockouts, and dissatisfied customers.

The Competitive Twist: An application should *not just display data*—it must help make decisions. For example, when an urgent order requires 10 units but only 7 are available while another lower-priority order requires 5 units, what should the system decide?

1.2.2 Our Solution: Decision-Driven Flow Command

Instead of simple CRUD tables, WarehouseIQ implements an active **Exception → Decision → Resolution** workflow:

1. **Inventory Radar & Anomaly Detection:** Real-time monitoring of SKU stock levels, identifying impending stockouts and damaged goods before they halt fulfillment.
2. **Intelligent Allocation & Prioritization:** Automated ranking of order queues by SLA urgency, carrier cut-off times, and revenue impact. When stock is constrained, the system evaluates split-allocation options and prompts operators with 1-click approval actions.

3. **Shift-Specific Workspaces:** Complete operational isolation between Shift A (Morning Inbound), Shift B (Afternoon Peak Fulfillment), and Shift C (Night Replenishment). Changing shifts alters all command desk metrics, task queues, and dispatch schedules.
 4. **AI Prediction Copilot:** A project-aware conversational chatbot integrated directly into the command suite, capable of answering operational questions instantly.
-

1.3 2. Complete Architecture & File Structure

To maintain clean code separation and prevent the monolithic clutter common in rapid prototypes, WarehouseIQ is organized into a modular full-stack architecture:

```
smart-warehouse-operations/
├─ client/
│   └─ src/
│       └─ components/          # Modular UI components
│           └─ AppShell.tsx     # Sidebar, persistent rail, shift selector, and
Sudheer account panel
│           └─ CommandDesk.tsx  # High-level operational metrics and signal feeds
│           └─ OrderQueueWorkspace.tsx # Order prioritization, allocation, and
split-view
│           └─ Workspaces.tsx   # Inventory Radar, Pick & Pack, Dispatch,
Analytics, Order Drawer
│           └─ AiPredictionWorkspace.tsx # Project-aware AI chatbot and decision
intelligence
│       └─ data/
│           └─ warehouseData.ts # Shift-specific datasets (Shift A, B, C)
│       └─ types/
│           └─ warehouse.ts     # TypeScript domain models and interfaces
│       └─ lib/
│           └─ warehouseAssistant.ts # Deterministic knowledge layer and fallback
engine
│       └─ pages/
│           └─ Home.tsx         # Main dashboard orchestrator
├─ server/
│   └─ _core/                  # Framework plumbing, Vite bridge, LLM proxy helper
│   └─ routers/
│       └─ ai.ts               # Secure server-side tRPC AI procedure (`ai.ask`)
│       └─ routers.ts          # Root tRPC router registry
│       └─ ai.ask.test.ts      # Vitest unit regression test for AI procedure
├─ drizzle/                   # Database schema and migration definitions
└─ PROJECT_DOCUMENTATION.md   # Detailed technical README
```

1.4 3. Technology Stack & Tooling Breakdown

Layer / Domain	Technology / Library	Exact Purpose & Role in WarehouseIQ
Frontend UI	React 19 & Tailwind CSS 4	Modern component rendering, responsive grid layouts, and custom design tokens.
Design System	shadcn/ui & Lucide Icons	Accessible UI primitives (dialogs, drawers, badges) and professional industrial iconography.
Client Routing	Wouter 3.3	Lightweight, hook-based client-side routing and URL state management.
Full-Stack API	tRPC 11 & Express 4	Type-safe RPC procedures connecting frontend clients to server-side business logic without boilerplate REST code.
Database & ORM	MySQL 8 / TiDB & Drizzle ORM	Relational data persistence, schema definitions, and query helper abstractions.
AI Integration	Manus Built-in LLM Proxy (BUILT_IN_FORGE_API_KEY)	Server-side secure AI question answering without exposing credentials in client-side bundles.
Testing & Build	Vitest & TypeScript (tsc)	Automated unit testing (ai.ask.test.ts) and zero-error compilation checks.
Version Control	Git & GitHub CLI (gh)	Continuous code synchronization to saiusudheernallabarika77-alt/Smart-Warehouse-Operations.

1.5 4. End-to-End Execution Workflow & Code Flow

1.5.1 Step 1: Initialization & Shift Context Loading

When the user opens the application (`Home.tsx`), the active shift (Shift A, Shift B, or Shift C) is loaded into reactive state. Changing the shift triggers `warehouseData.ts` to supply shift-specific records:

- **Shift A (06:00 - 14:00):** Focuses on inbound container receiving, put-away validation, and morning dock clearance.
- **Shift B (14:00 - 22:00):** Focuses on peak afternoon order velocity, picking waves, quality-check bottlenecks, and carrier cut-off compliance.
- **Shift C (22:00 - 06:00):** Focuses on night-shift replenishment, cycle counting, system reconciliation, and early morning dispatch preparation.

1.5.2 Step 2: Order Queue & Split-Allocation Decision Flow

1. Operators view the active order queue in `OrderQueueWorkspace.tsx`.
2. Each order displays its SLA status, priority tier (Urgent, Standard, Economy), and inventory allocation state.
3. If an order lacks sufficient stock, clicking the order opens the `OrderDrawer.tsx`, exposing automated AI allocation recommendations (e.g., partial split-shipment vs. holding for batch replenishment).
4. Approving the recommendation instantly updates inventory levels and transitions the order to the Pick & Pack work queue.

1.5.3 Step 3: Secure AI Copilot Chatbot Flow

When a user asks a question in the AI Prediction module (`AiPredictionWorkspace.tsx`):

1. **Client Request:** The chat interface packages the user prompt along with serialized active shift context (orders, inventory levels, bottlenecks).
2. **Server-Side Proxy (`server/routers/ai.ts`):** The tRPC procedure `ai.ask` receives the payload. It constructs an expert warehouse operations system prompt.
3. **Secure LLM Invocation:** The server calls the Manus built-in LLM proxy using `BUILT_IN_FORGE_API_KEY`. **No API key is ever sent to or exposed in the browser.**
4. **Fallback Resilience:** If network latency occurs, `warehouseAssistant.ts` immediately executes its deterministic knowledge layer to return a precise, structured response with a “Local project fallback active” status indicator.

1.6 5. Mentor & Evaluator Q&A Master Brief

During your hackathon presentation or mentor review, use these exact, confident explanations for anticipated questions:

1.6.1 Q1: “Is this connected to a real physical warehouse?”

Answer: “No, the hackathon brief explicitly stated that real-world APIs or warehouse hardware are not required. Instead, we built a high-fidelity WMS simulation using robust mock datasets and shift-aware states that behave identically to an enterprise fulfillment center like Amazon FC or DHL.”

1.6.2 Q2: “How do shifts differ if the data is mocked?”

Answer: “Each shift has a dedicated operational profile in `warehouseData.ts`. Shift A focuses on inbound receiving; Shift B handles peak afternoon order velocity and quality-check queues; Shift C

manages night replenishment and dispatch cut-off clearing. Changing the shift instantly transforms command desk metrics, active order queues, and AI recommendations.”

1.6.3 Q3: “Which API key was used, and how is it secured?”

Answer: “We did not hardcode any personal OpenAI or Anthropic API keys in the frontend code, which would be a severe security risk. Instead, we implemented a secure server-side tRPC procedure (`ai.ask`) that communicates with the Manus built-in LLM service using server-side environment variables (`BUILT_IN_FORGE_API_KEY`). Furthermore, we built a deterministic fallback engine in `warehouseAssistant.ts` so the chatbot remains 100% operational even offline.”

1.6.4 Q4: “How is the code structured to prevent clutter?”

Answer: “We refactored the codebase from a monolithic file into a clean, modular architecture: `types/` for domain models, `data/` for shift datasets, `components/` for modular UI pieces (`CommandDesk`, `OrderQueueWorkspace`, `Workspaces`, `AiPredictionWorkspace`), and `server/` for backend routes and tests.”

1.7 6. References & Verified Deliverables

1. **Live Deployment:** [WarehouseIQ Live App](#) [1]
2. **GitHub Repository:** [Smart-Warehouse-Operations on GitHub](#) [2]
3. **Technical Documentation:** `PROJECT_DOCUMENTATION.md` in repository root [3]

[1] WarehouseIQ Live Preview URL. Available online at: <https://smartwarehouse-nwqedban.manus.space>

[2] GitHub Repository. Available online at: <https://github.com/saisudheernallabarika77-alt/Smart-Warehouse-Operations>

[3] Project Technical Documentation. Available in repository root as `PROJECT_DOCUMENTATION.md`.